GenAI, RAG, Agentic AI

GenAI:

1. Lang chain: What is chain, lang graph, lang smith, lang chain

2. lang chain eco-system

3. what is RAG, types of RAGs, components of RAG

4. what is chunking, types of chunking

5. vector embedding, types of vector embedding models

6. vector database, types of vector database

Online pipeline:

7. query decomposition and why we use it?

8. chunk re-ranking, techniques in re-ranking

9. what is output template

10. What is tools, why do we use tools

11. what's memory, why use, memory techniques, how is it optimized

12. what is agent?

13. summarized memory, session memory, Parallel processing or serial processing?

Langchain eco-system:

langchain

It's a frame work that makes building LLM powered applications easy. An LLM (like GPT) is the brain" and LangChain is the framework that helps connect the brain to tools, data, memory, and workflows.

The three main steps/components in langchain:

i) Retrieve ii) summarize iii) Answer/Output

whether you generate any kind of generative AI applications or AI assistant these 3 components will be there.

i) Retrieve:

- Data Ingestion (Document loader)
- “Text splitter” is a separate package that is available in langchain where there are multiple ways of splitting the text.
- Every LLM has a limitation w.r.t context called “context window” .
- So using text splitter we divide the data in smaller chunks.
- After dividing the data we need to store it somewhere so, we store it in “Vector database”.
- Within this we also use “vector embeddings” which is used to convert text into vectors.
- We store it inorder to do different different kind of search like (symmantic, graph DB).
- With the help of this search we will be able to retrieve right amount of context and give it to our LLM to finally generate the output.

ii) Summarize:

- In this we follow a sequential order. It means that the execution of any task will happen sequentially. We cannot go back.
- The langchain follows a DAG graph **(Directed Acyclic Graph)** which follows sequential task execution.
- Three important components in Summarize is **(Prompt, LLM, Context)**
- We implement a “chaining concept”. Where we give prompt → Integrate the prompt with LLM → Context that is supplied to the LLM.
- The execution happens sequentially, after that whatever the output we get it’s transferred to the the next step/component **(Answer/output).**

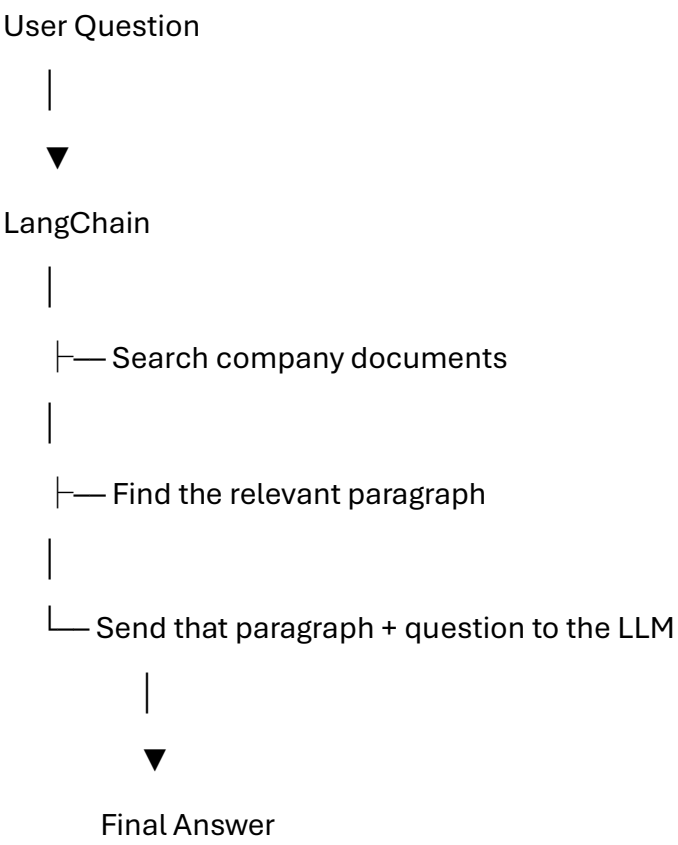
iii) Answer/Output:

- Here we can use multiple components. One is “memory” like “Persistent memory”.
- Finally we generate the output

Ex: customer support chatbot: When a user asks:

"What's the refund policy?"

The flow looks like:



LangGraph:

The main aim is to create stateful multi AI agentic application.

- We create multiple AI agents which will do separate tasks and they can communicate with each other to solve a complex workflow.
- It need not be a directed or cyclic graph and it need not be sequential, this can follow different ways.

Three important components of langgraph:
i) Tasks ii) Nodes iii) Edges iv) Graph

- The complex workflow defined will be in a form of a graph.
- A feedback mechanism can be generated and this way you are able to solve a complex workflow.
- Each task is executed by a separate AI agent. An output of one task can be passed to the output of another task.
- "Persistent memory" is more efficient and memory is shared between each and every task/node.

LangSmith:

Langsmith is a platform designed to streamline the development, testing, debugging and monitoring of LLM apps, offering tools for observability, evaluation and prompt engineering.

Why use Langsmith?

There are three important reasons:

i) Performance monitoring:

It tracks API calls, token usage, errors and latency.

ii) Debugging and Evaluation:

It ensures models return expected results and avoid unnecessary API costs.

iii) Independent framework:

what is RAG, types of RAGs, components of RAG?

RAG:

Retrieval-Augmented Generation (RAG) is an AI framework that fetches authoritative, external data to enhance Large Language Model (LLM) responses.

Core RAG Components:

A RAG pipeline has 2 primary phases **i) Preprocessing and ii) Inferencing.** This has 4 fundamental steps.

1. External Knowledge Base (Ingestion): The repository of authoritative data (e.g., company wikis, PDFs, or private databases). The raw data is broken into smaller pieces (**chunking**) and translated into mathematical representations (**embeddings**).

2. Retrieval Module: When you ask a question, the system matches your query to the most similar document embeddings in the knowledge base using vector or keyword searches.

3. Augmentation: The system dynamically merges your original question with the retrieved, context-rich data to construct a highly specific Prompt Template.

4. Generator: The LLM processes the augmented prompt to synthesize a grounded, accurate response.

Types of RAG Architectures:

1. Simple RAG: The original, basic architecture. Converts a query to embeddings, searches a vector database in a single step, and passes the result to the LLM.

2. Advanced RAG: Optimizes retrieval by utilizing techniques like better chunking strategies, document re-ranking, and context filtering before feeding data to the LLM.

3. Agentic RAG: Uses a dynamic AI agent that can plan, break down complex multi-step questions, and independently search various tools or APIs to retrieve the precise information needed.

4. GraphRAG: Integrates Knowledge Graphs to connect ideas and retrieve structured relationships between concepts (e.g., how symptom A relates to disease B).

5. Multimodal RAG: Expands the knowledge base and model comprehension beyond text to include images, audio, and video.

6. Adaptive RAG: Intelligently learns from user patterns and changes its retrieval methodology (e.g., swapping from keyword to vector search) based on context.

Chunking:

Chunking is the process of breaking down large pieces of text or data into smaller, manageable segments called "chunks". It is a foundational step in building AI applications like Retrieval-Augmented Generation (RAG).

Types of Chunking:

1. Fixed-Size Chunking: Splits text into uniform blocks based on a predefined number of characters or tokens.

2. Recursive Chunking:

Uses a hierarchical list of predefined separators (like paragraphs, sentences, and spaces) to break down text. If the first split is too large, it recursively moves to the next separator until the ideal chunk size is achieved.

3. Semantic Chunking:

Analyzes the actual meaning and context of the text rather than just counting words. It groups sentences based on semantic similarity and starts a new chunk whenever the topic shifts.

4. Document-Structure Chunking:

Breaks text down based on the logical layout of the document. For example, it will treat markdown headers, specific sections, or tables as natural chunk boundaries rather than forcing arbitrary limits.

5. Agentic Chunking:

Employs an LLM to act as an agent that reasons through the text. The AI dynamically reads, understands, and segments the text based on its content, while potentially assigning summaries and metadata to each chunk.

Why the Choice of Chunking Matters?

Choosing the right method directly impacts your AI's performance. If chunks are too small, the AI will lack the background context needed to answer a query. If chunks are too large, they dilute the significance of specific details and exceed the token limits of the model.

Vector Embeddings:

Vector embeddings are mathematical representations of data (text, images, audio) as sequences of numbers (vectors). Vector embedding models generally fall into several distinct categories depending on the data type they process and how they evaluate context.

1. Text & Natural Language Models:

- **Transformer-Based Models:** The modern standard. Models process entire sentences or documents simultaneously, producing context-aware vectors.
- **Word Embeddings:** Older, foundational models that map individual words to a static vector. Because they are static, they do not understand context (e.g., they treat "bank" identically in "river bank" and "money bank").
- **Sentence/Document Embeddings:** Models explicitly optimized to compress a whole paragraph, sentence, or document into a single vector for search and retrieval.

2. Computer Vision & Image Models:

- **Image Embeddings:** These models translate visual features like color, shapes, and textures into spatial vectors. They are heavily used in visual search engines and facial recognition.

3. Multimodal Embeddings: Advanced models that map entirely different data types such as text and images into the same mathematical space. This allows users to search for an image using a text query without needing a translation step.

4. Specialized Data Models:

- **Audio Embeddings:** Capture features such as rhythm, tone, and pitch for voice analysis, music recommendation and audio search.
- **Graph Embeddings:** Designed to represent nodes and connections (edges) in relational networks (like social networks or fraud detection) to predict relationships.

Vector Database:

A vector database is a specialized data management system designed to store, index and query high-dimensional vector embeddings. Vector databases use mathematical algorithms to locate data points based on semantic or conceptual similarity.

Core Mechanics:

- **Embeddings**: Raw data (text, video, audio) is converted into numerical strings.
- **Distance Metrics**: Similarity is calculated via formulas like Cosine Similarity or Euclidean Distance.
- **ANN Algorithms**: High-speed search relies on Approximate Nearest Neighbor (ANN) calculations.

1. Pure (Purpose-Built) Vector Databases:

These systems are built from scratch specifically to handle high-dimensional vector search. They decouple infrastructure layers to provide ultra-fast throughput and high scalability.

2. Integrated (Multi-Model) Vector Databases:

These are existing relational, NoSQL or search platforms that add vector storage and indexing capabilities through updates or plugins.

Vector Databases for RAG Pipelines:

- **Pinecone:** A fully-managed cloud vector DB. Known for enterprise-grade reliability and scalability.
- **MongoDB:** A document database with built-in vector search. Supports hybrid full-text and embedding search, as well as rich JSON filtering.
- **Qdrant:** An open-source Rust vector database. Excels at real-time embedding search with rich JSON-based payload filtering.

Query decomposition:

Query decomposition is an AI technique that breaks a complex, multifaceted user question into smaller, independent sub-questions.

Why We Use It?

- **Improves Search & Retrieval:** When a user asks a complex question, a direct search in a database often returns irrelevant, diluted, or noisy results. Decomposing the query allows the AI to search for highly targeted keywords for each sub-question.
- **Handles Hidden Multi-Part Questions:** Many user questions are secretly multiple questions in disguise (e.g., *"Did Microsoft or Google make more money last year?"*). Decomposition forces the AI to break this down into *"How much did Microsoft make?"* and *"How much did Google make?"* before comparing the two.
- **Reduces Hallucinations:** By forcing the model to process information incrementally and verify smaller facts first, you give the AI a built-in reasoning step. This drastically reduces the likelihood of generalized, made-up answers.

- **Overcomes Context Length Limits:** Instead of trying to retrieve the answer for one massive, confusing question, the AI retrieves compact chunks of relevant information step-by-step.

chunk re-ranking, techniques in re-ranking?

Chunk re-ranking is a crucial two-stage Retrieval-Augmented Generation (RAG) technique that prioritizes precision over speed.

what is output template?

An output template dictates the precise structure, format and schema of the AI's final response. It acts as a set of rules (often defining JSON fields, Markdown styles, or text delimiters) to ensure the generated AI content is perfectly formatted for downstream applications or user interfaces.

what is AI agent?

An AI agent is an autonomous software system that uses artificial intelligence to perceive its environment, make decisions and take independent actions to achieve specific goals.